LLMsVerifier

LLMsVerifier

Tagline: Verify. Monitor. Optimize.

Summary: LLMsVerifier is an enterprise-grade platform for verifying, monitoring, and optimizing Large Language Models across many providers, built on a mandatory “Do you see my code?” verification test so that only models proven to actually work are ever marked usable or exported.

Short description: A Go platform that verifies, benchmarks, monitors, and optimizes LLMs across multiple providers. Every model must pass a mandatory code-visibility test before use; it then runs latency, streaming, function-calling, vision, and embedding checks, and exports verified-only configurations for AI CLI tools. (~40 words)

Long description: LLMsVerifier is a comprehensive platform for verifying, monitoring, and optimizing LLM performance across multiple providers. Its core principle is *mandatory verification*: before any model is marked usable — or included in an exported configuration — it must affirmatively pass a “Do you see my code?” test, making real HTTP calls to the provider and analyzing the response for genuine comprehension. Beyond that gate, the Verifier Engine runs a battery of capability tests (existence, responsiveness, latency, streaming, function calling, vision, embeddings), and a Reporter Engine produces markdown and JSON reports. The system is modular and event-driven, exposing CLI, TUI, Web, and REST API interfaces over a core of Verifier Engine, Reporter Engine, and Configuration Manager, plus advanced layers: a Supervisor/Worker pattern for LLM-powered task decomposition, sliding-window + LLM-summarization context management for very long sessions, cloud-backed checkpointing, and a failover system with circuit breakers and latency-based routing. Infrastructure includes a pub/sub event bus, cron scheduling, pricing/limits detection, a vector database for RAG, and an export system. A signature branding convention appends an `(llmsvd)` suffix to every generated provider/model so verified outputs are traceable, and only verified models are written into exported configs for AI CLI tools such as OpenCode, Crush, and

Claude Code. It ships production tooling: Docker/Kubernetes/ Helm deployment, Prometheus/Grafana monitoring, LDAP/SSO, and SQLCipher-encrypted storage.

Why we built it: Because configuration-only checking is unreliable — an API key can expire, a model can be deprecated, and a config file tells you nothing about real latency, real errors, or whether the model can actually see and understand your input. LLMsVerifier replaces "it's in the config, so it must work" with proof: only models that demonstrably respond correctly are marked usable and exported.

Why it's a game-changer: It makes LLM fleets *trustworthy*. Instead of hoping a configured model works, teams get an enforced, testable guarantee that every model in play has passed real verification — with monitoring, failover, and verified-only export closing the loop. Within the Helix ecosystem it serves as the single source of truth for LLM model, provider, and verification metadata that other services (e.g. HelixTranslate) route against.

What's innovative: - **Mandatory "Do you see my code?" verification** — a real, HTTP-backed comprehension gate that a model must pass before it is usable (the product's signature differentiator). - **Verified-only configuration export** — generated configs for AI CLI tools contain only models that passed verification. - **`(llmsvd)` branding-suffix system** — every generated provider/model carries a traceable suffix. - **Capability detection** across many CLI agents and providers — streaming types (SSE, WebSocket, JSONL, EventStream), compression, and caching behaviors. - **Resilient failover** — circuit breakers, latency-based routing (fail over when time-to-first-token exceeds a threshold), health probes, and weighted traffic. - **Long-running autonomy** — a Supervisor/Worker pattern plus checkpointing and memory integration for extended sessions. - **RAG / vector-DB integration** for context enhancement.

Biggest technical challenges + how solved: - **Proving a model actually works, not just that it's configured.** Solved by the mandatory code-visibility test making real API calls and analyzing responses for affirmative comprehension, plus a broad suite of capability tests — and by exporting only verified models. - **Reliability across many flaky third-party providers.** Solved with a failover orchestrator: circuit breakers (mark degraded after N failures in M seconds), latency-based routing, periodic health checks, and weighted routing between cost-effective and premium models. - **Sustaining very long, autonomous sessions.** Solved with the Supervisor/Worker decomposition pattern, periodic checkpointing to cloud storage, and context management (sliding

window + LLM summarization + RAG). - **Provider sprawl.** Many per-provider Go adapters behind a common interface, with real endpoints enumerated centrally.

Tech stack (why + how): - **Go** — the core platform language; multi-threaded Verifier Engine and services. - **Gin** — REST API server (JWT auth, rate limiting, WebSocket/SSE). - **SQLite + SQLCipher** — embedded storage with database-level encryption for sensitive verification data. - **Redis** — caching layer. - **RabbitMQ + Kafka** — messaging/streaming for the event-driven architecture. - **gRPC + Protocol Buffers** — inter-service communication and event transport. - **QUIC / HTTP-3 (quic-go)** — modern transport support (repo docs flag HTTP/3 provider availability as limited — treat as capability, not universal claim). - **JWT + LDAP/NTLM** — enterprise authentication (SSO/SAML/OIDC claimed in docs). - **Viper (config), Logrus (logging), Brotli/compress (compression)** — operational plumbing. - **Angular** — the Web single-page application. - **Python + JavaScript SDKs** — client access with OpenAPI/Swagger docs. - **Docker, Kubernetes, Helm** — production deployment with health monitoring and autoscaling. - **Prometheus + Grafana** — metrics and dashboards. - **Testify (Go) + node –test/jsdom (web)** — layered testing.

Public links: - Product repo, PUBLIC: `github.com/vasic-digital/LLMsVerifier` - Issues / discussions, PUBLIC: `github.com/vasic-digital/LLMsVerifier/issues` and `/discussions` - **Not public / internal:** local Web UI + Swagger (`http://localhost:8080`, `/swagger/index.html`); private mirror upstreams configured via `install_upstreams.sh` (GitHub/GitLab/GitFlic/GitVerse). Ignore the `package.json` `your-org` placeholder URL and the lowercase `llm-verifier` Docker-label URL — treat only the `vasic-digital/LLMsVerifier` repo as canonical.

Suggested diagrams/illustrations (OpenDesign): 1. **Mandatory verification gate** — a model entering a gate labeled “Do you see my code?”; PASS → marked usable + `(llmsvd)` suffix + eligible for export; FAIL → rejected (never exported). The signature visual. 2. **Verification test matrix** — a grid of capability checks (existence, responsiveness, latency, streaming, function calling, vision, embeddings) across provider columns, with pass/fail cells. 3. **Failover orchestration** — provider chain with circuit-breaker states (closed/open/half-open), latency-based rerouting, and weighted traffic split. 4. **Verified-only export flow** — verified model pool → config generator → AI CLI tool configs (OpenCode / Crush / Claude Code), with unverified models visibly filtered out.

Site relevance: Both. Strong product page on **vasic.digital** (position as the LLM-trust/verification layer of the Helix LLM cluster); portfolio entry on **milosvasic.ru**. (Repo lives in the `vasic-digital` org, but functionally it is part of the Helix LLM-infrastructure cluster.)

Priority tier: Helix-primary (LLM-infrastructure cluster; single source of truth for LLM/provider/verification metadata). Ranks after HelixTrack.

Source provenance: Local repo `/Volumes/T7/Projects/llms_verifier` — `go.mod`, `Dockerfile`, `docs/ARCHITECTURE_OVERVIEW.md`, `docs/CAPABILITY_DETECTION.md`, `OPTIMIZATIONS.md`, `llm-verifier/README.md`, `llm-verifier/verification/code_verification.go`, `llm-verifier/providers/model_verification_service.go` + `MODEL_VERIFICATION_README.md`, `helix-deps.yaml`, `CLAUDE.md`/`CONSTITUTION.md`; plus harvested `_analysis/top20/LLMsVerifier.readme.txt` and `_analysis/MASTER-inventory.md`. Caution flags (verify or soften): provider count — README says “12 adapters” but the providers directory + `provider_mapping.txt` list ~26 (use “12+ / more in progress”); one legacy doc (`VERIFICATION_HOW_IT_WORKS.md`) describes verification as config-only/aspirational, but the Go source implements real HTTP verification — trust the code; license MIT (README) vs Apache-2.0 (Dockerfile label) discrepancy; many aspirational “FINAL/COMPLETE/ULTIMATE” status .md files exist — prefer code/docs/go.mod as authoritative. `llm_orchestrator`/`llm_provider`: LLMsVerifier is their upstream source-of-truth (they read its model metadata), but the LLMProvider dependency was dropped from this repo’s own module graph; its own external own-org dep is `Challenges`. “Helix-Flow” appears only as an org name in governance lists — not a component.